

Chapter 3

Data Modeling Using the Entity-Relationship (ER) Model

1

1

Chapter Outline

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
 - Entities and Attributes
 - Entity Types, Value Sets, and Key Attributes
 - Relationships and Relationship Types
 - Weak Entity Types
 - Roles and Attributes in Relationship Types
- ER Diagrams – Notation
- ER Diagram for COMPANY Schema
- Alternative Notations – Unified Modeling Language (UML) class diagrams, others

2

2

Overview of Database Design Process

- Two main database-related activities:
 - Database design
 - DB application programs design
- Focus in this chapter is on the design of the conceptual schema in Entity Relationship (ER) model for database design.

3

3

Main Phases of Database Design

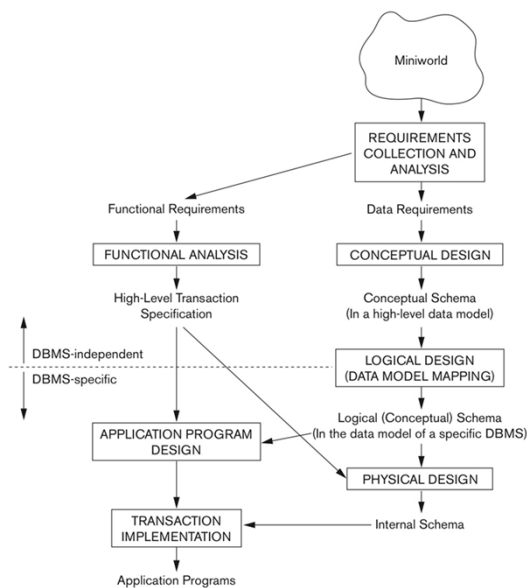


Figure 3.1 A simplified diagram to illustrate the main phases of database design.

4

4

Conceptual data model (DBMS-independent model) vs. Implementation data model.

- Each implementation data model (DBMS-specific model) is directly support by a group of DBMSs.
- **Conceptual data model** is more powerful and certain constraints (e.g., cardinality ratio constraint and participation constraint) can be easily specified, such that **allows more accurate modeling**.
- **Conceptual data model** is easy to understand, check, update and communicate with users.
- Moreover, **conversion from a conceptual data model to an implementation data model is straightforward**, so it is not a big overhead.

5

5

Example COMPANY Database

- We need to create a database schema based on the following (simplified) **requirements** of the COMPANY Database:
 - The company is organized into DEPARTMENTS.
 - Each department has a unique number and a name, and an employee who *manages* the department. We keep track of the start date of the department manager.
 - A department may have several locations.
 - Each department *controls* a number of PROJECTs.
 - Each project has a unique name, a unique number and is located at a single location.
 - Each project is controlled by one department.

6

6

Example COMPANY Database (cont'd)

- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
 - Each employee *works for* one department but may *work on* several projects.
 - We keep track of the number of hours per week that an employee currently works on each project.
 - We also keep track of the *direct supervisor* (i.e., immediate supervisor) of each employee.
- Each employee may *have* a number of DEPENDENTS.
 - For each dependent, we keep track of his/her name, sex, birthdate, and relationship to the employee.

7

7

ER Model Concepts

- Entities and Attributes
 - Entities are specific objects or things in the mini-world that are to be represented in the database.
 - e.g., the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
 - Attributes are properties used to describe an entity.
 - e.g., an EMPLOYEE entity may have attributes, such as Name, SSN, Address, Sex, and Birthdate.
 - A specific entity will have a value for each of its attributes.
 - e.g., a specific employee entity may have **Name**='John Smith', **SSN**='123456789', **Address**='731, Fondren, Houston, TX', **Sex**='M', **Birthdate**='1980-01-18'
 - Each attribute has a *value set* (or data type) associated with it: e.g., integer, string, float, enumerated type, ...

8

8

Types of Attributes

- **Simple (or Atomic) attribute**
 - An attribute that is not divisible. For example, SSN, Sex, etc.
- **Composite attribute**
 - An attribute that is composed of several components. For example, **Address**(House#, Street, Apt#, City, State, ZipCode, Country), **Name**(First Name, Middle Name, Last Name), etc.
 - Composition may form a hierarchy where some components are composite attributes by themselves.
- **Single-valued attribute**
 - An attribute which has a single value for a particular entity.
- **Multivalued attribute**
 - may have multiple values for an entity. For example, Color of a CAR and Previous_Degree of a STUDENT.
 - Denoted as {Color} or {Previous_Degree}.
- **Derived attribute**
 - An attribute whose value can be derived from other attribute values. For example, Age of an employee is derived from the date of birth (DOB).

9

9

Hierarchy of Composite Attributes

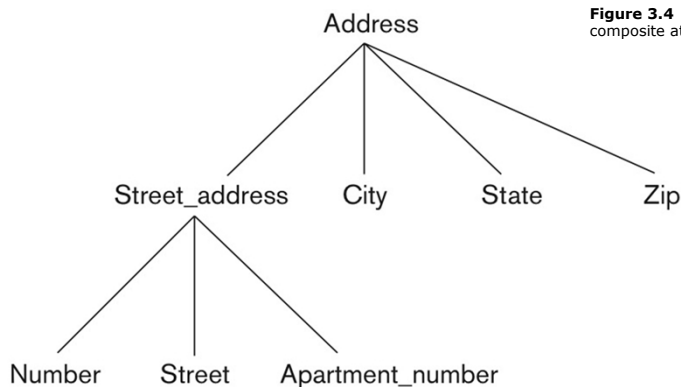


Figure 3.4 A hierarchy of composite attributes.

10

10

Composite Multivalued Attribute

- If an attribute is not only composite but also multivalued, it is called a **composite multivalued attribute**.
 - For example, PreviousDegree of a STUDENT is a composite multivalued attribute denoted by **{PreviousDegree(College, Year, Degree, Field)}**
 - Multiple PreviousDegree values can exist for some student.
 - Each PreviousDegree value has four component attribute values: College, Year, Degree, and Field
- The content of a composite multivalued attribute is like a table, and a composite multivalued attribute may be nested to a number of levels.

11

11

Entity Types and Key Attributes

- For a set of real-world entities that have the same attributes, we can define an entity type.
 - For example, EMPLOYEE and PROJECT entity types.
- Each entity type is described by a name and a list of attributes. For example,
 - CAR** (Registration(Number, State), Vehicle_id, Make, Model, Year, {Color})
 - EMPLOYEE** (SSN, Fname, Lname, Birthdate, Sex, Address, Salary, Dept_number)
- A **key attribute** (of each entity type) has different values for different entities (of that entity type), so its value can be used to identify an entity uniquely.
 - For example, SSN is a key attribute of EMPLOYEE entity type.

12

12

Entity Types and Key Attributes (cont'd)

- A key attribute may be a composite attribute.
 - Registration(Number, State) is a key attribute of the CAR entity type.
- Some entity type may have more than one key attribute.
 - e.g., the CAR entity type may have two key attributes:
 - Registration(Number, State)
 - Vehicle_id
- Each key attribute name is underlined in the ER diagram.

13

13

Displaying an Entity type

- In ER diagrams, an entity type is displayed in a rectangular box.
- Attributes are displayed in ovals.
 - Each attribute is connected to the rectangular box representing its entity type.
 - Components of a composite attribute are connected to the oval representing the composite attribute.
 - Each key attribute is underlined.
 - So, if a combination of attributes is a key, it must be represented as a composite attribute, like Registration(Number, State). If we underline Number and State, it means Number is a key and State is another key.
 - Multivalued attributes are displayed in double ovals.
 - PreviousDegree(College, Year, Degree, Field) is multivalued, only PreviousDegree is in double oval, while its components are in a single oval.

14

14

Entity Type CAR with two keys and a corresponding Entity Set

(a)

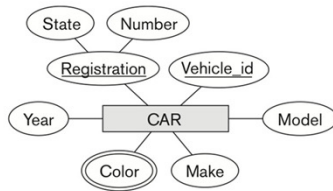


Figure 3.7 The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR₁
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

15

15

Entity Set

- Each entity is an instance of the corresponding entity type.
 - Each entity stores the attribute values of a real-world entity, so there is one-to-one correspondence between an entity and a real-world entity.
- An entity set is the current set of entities of the corresponding entity type that are stored in the database.
- Previous slide shows three CAR entities in the entity set for CAR.

16

16

Initial Design of Entity Types for the COMPANY Database Schema

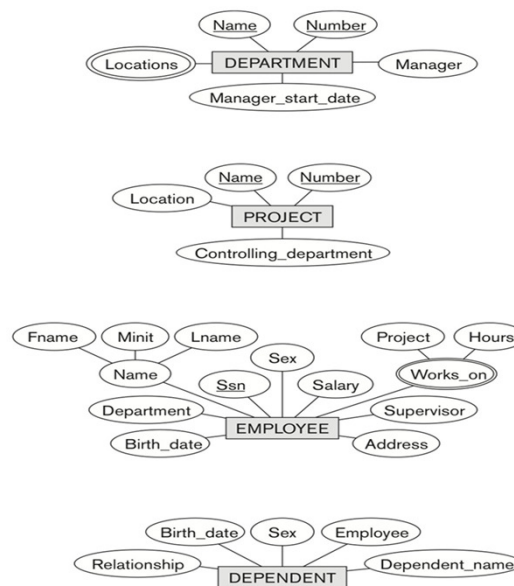
- Based on the requirements for the COMPANY database, we can identify four initial entity types:
 - DEPARTMENT
 - PROJECT
 - EMPLOYEE
 - DEPENDENT
- The attributes of initial entity types are derived from the description of the data requirements.

17

17

Initial Design of Entity Types: DEPARTMENT, EMPLOYEE, PROJECT, and DEPENDENT

- In these initial entity types, there are **reference attributes** that refer to other entities (of the same or different entity type).
- Each reference attribute should be replaced by a relationship type.



18

18

Refining the Initial Design by Introducing **Relationship Types**

- ER model has two main concepts (i.e., modeling constructs):
 - **Entity types**
 - **Relationship types**
- Some aspects in the requirements can be represented as **relationships** between entities.
- When an attribute of one entity type refers to another entity type, some relationship exists.

19

19

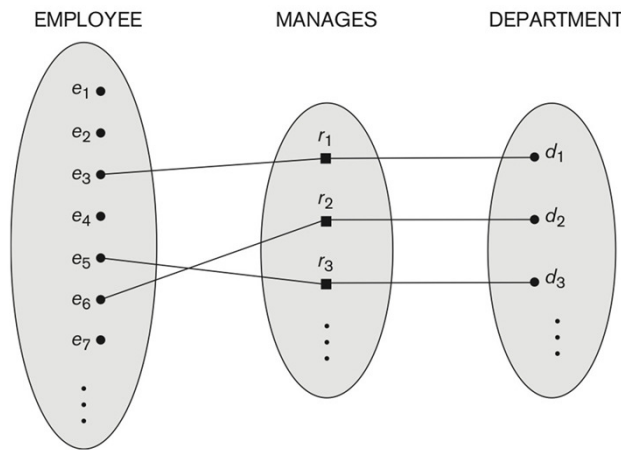
Relationships and Relationship Types

- A **relationship** is an association between (two or more) entities. For example,
 - EMPLOYEE Franklin Wong *manages* the Research DEPARTMENT.
 - EMPLOYEE John Smith *works on* the ProductX PROJECT.
- A **relationship type** is defined for a set of relationships of the same type. For example,
 - WORKS_ON relationship type in which EMPLOYEE and PROJECT participate.
 - MANAGES relationship type in which EMPLOYEE and DEPARTMENT participate.
- The **degree of a relationship type** is the number of participating entity types.
 - Both MANAGES and WORKS_ON are *binary* relationship types.

20

20

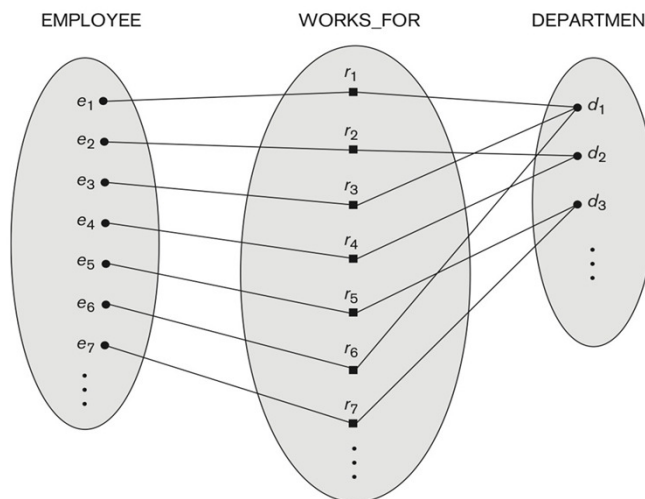
Instances of the 1:1 **MANAGES** Relationship Type (between DEPARTMENT and EMPLOYEE)



21

21

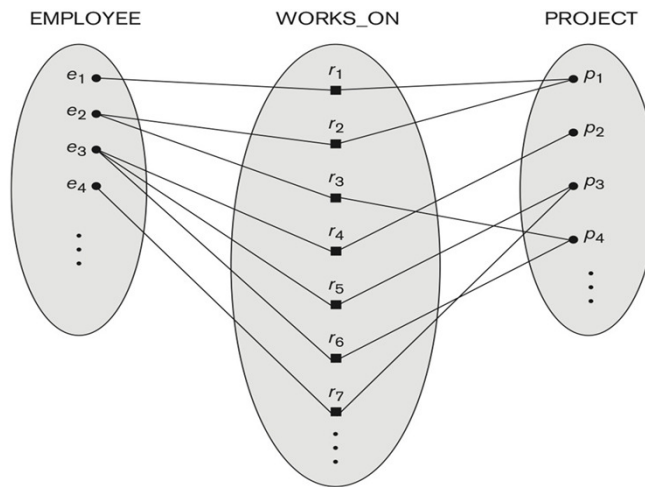
Instances of the 1:N **WORKS_FOR** Relationship Type (between DEPARTMENT and EMPLOYEE)



22

22

Instances of the M:N **WORKS_ON** Relationship Type (between EMPLOYEE and PROJECT)



23

23

Relationship Type vs. Relationship Set

- Relationship Type:
 - is the schema description of a relationship.
 - identifies the relationship name and the participating entity types.
 - also specifies certain relationship constraints.
- Each instance of the relationship type associates individual entities — one from each participating entity type.
- Relationship Set:
 - The current set of instances of a relationship type.

24

24

Relationship Type vs. Relationship Set (cont'd)

- In ER diagrams, we represent the *relationship type* as follows:
 - Diamond-shaped box is used to display a relationship type.
 - Connected to the participating entity types via (straight) lines.



25

25

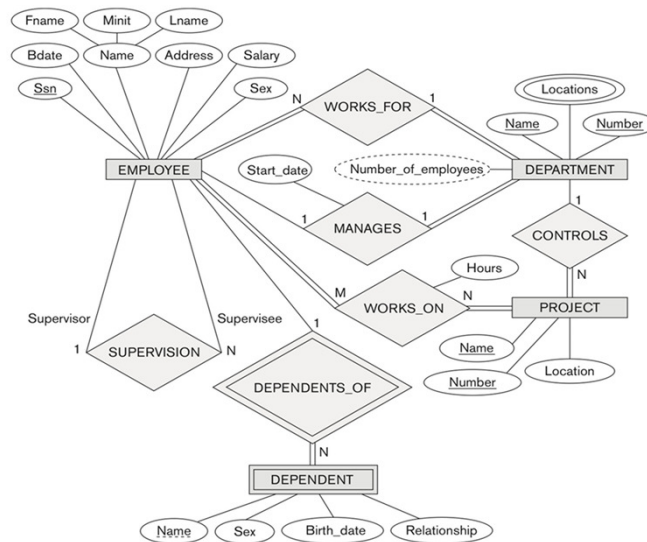
Refining the COMPANY Database Schema by Introducing Relationship Types

- By examining the requirements, six relationship types are identified. All are *binary* relationships.
- Listed below with their participating entity types:
 - **WORKS_FOR** (between EMPLOYEE and DEPARTMENT)
 - **MANAGES** (between EMPLOYEE and DEPARTMENT)
 - **CONTROLS** (between DEPARTMENT and PROJECT)
 - **WORKS_ON** (between EMPLOYEE and PROJECT)
 - **SUPERVISION** (between EMPLOYEE (as supervisor) and EMPLOYEE (as supervisee))
 - **DEPENDENTS_OF** (between EMPLOYEE and DEPENDENT)

26

26

Company ER DIAGRAM with WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF Relationship Types



27

27

Discussion on Relationship Types

- Reference attributes of the initial entity types are changed to relationships:
 - Manager attribute of DEPARTMENT → MANAGES
 - Works_on attribute of EMPLOYEE → WORKS_ON
 - Department attribute of EMPLOYEE → WORKS_FOR
 - Supervisor attribute of EMPLOYEE → SUPERVISION
 - Controlling_department attribute of PROJECT → CONTROLS
 - Employee attribute of DEPENDENT → DEPENDENTS_OF
- In general, more than one relationship type can exist between the same participating entity types.
 - MANAGES and WORKS_FOR are distinct relationship types between EMPLOYEE and DEPARTMENT.

28

28

Recursive Relationship Type

- A recursive relationship is a relationship between entities of the same entity type, playing different roles.
- Example: the SUPERVISION relationship type
 - Each instance of SUPERVISION relates two distinct EMPLOYEE entities:
 - one employee in *supervisor* role.
 - one employee in *supervisee* role.

29

29

Instances of a Recursive Relationship

- We may need to display **role names** to distinguish participations.

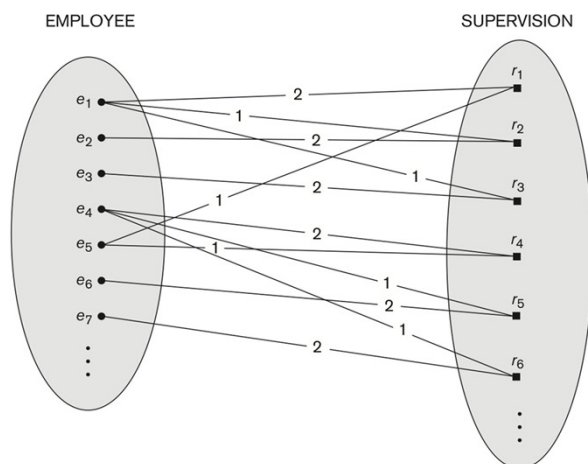


Figure 3.11 A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

30

30

Structural Constraints on Relationship Types

- **Cardinality Ratio Constraint** specifies the type of association:
 1. one-to-one (1:1)
 2. one-to-many (1:N) or many-to-one (N:1)
 3. many-to-many (M:N)
- **Participation Constraint** on each participating entity type:
 1. **partial participation**: participation in the instances of a relationship type is optional for each entity (i.e., participation is not mandatory).
 2. **total participation**: participation is mandatory for each entity (i.e., at least one participation is required).
- Both Cardinality Ratio Constraint and Participation Constraint are derived from the knowledge of mini-world constraints. 31

31

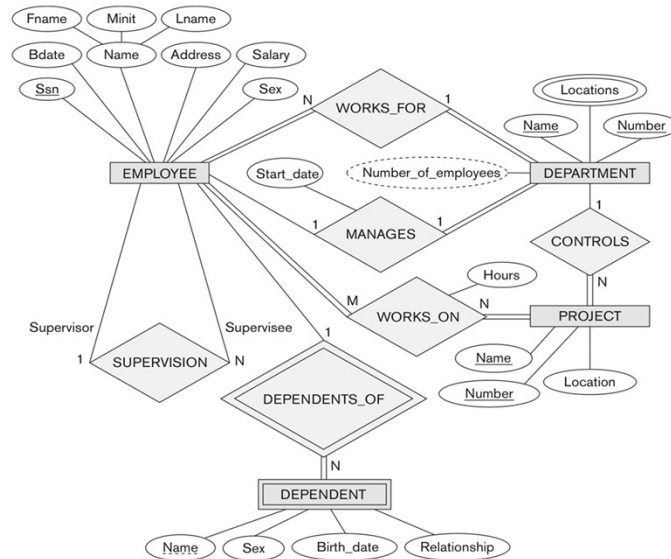
Notation for the Constraints on Relationship Types

- **Cardinality ratio** (of a binary relationship type): 1:1, 1:N, or M:N.
 - Shown by placing appropriate numbers on the relationship edges.
- **Participation constraint** (on each participating entity type): total or partial.
 - Total participation is shown by a double line (between the entity type and the relationship type), and partial participation is shown by a single line.

32

32

COMPANY ER Diagram with Structural Constraints on the Relationship Types



33

33

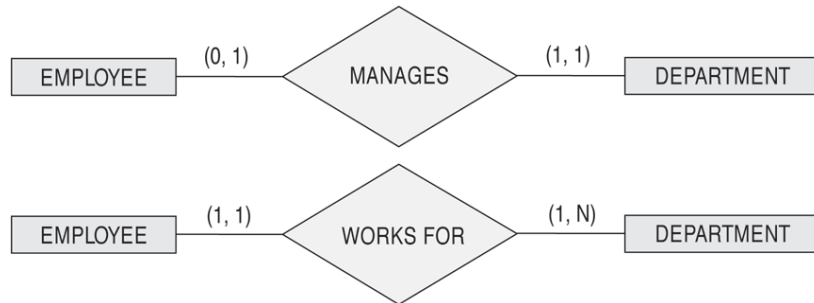
Alternative (min, max) Notation for Structural Constraints on Relationships

- Specified on each participation of an entity type E in a relationship type R, as (min, max)
 - Specifies that each entity *e* in E participates in at least *min* and at most *max* relationship instances in R.
 - Must have $\min \leq \max$, $\min \geq 0$, $\max \geq 1$
 - If $\min=0$, the participation constraint is partial. If $\min>0$, the participation constraint is total.
- Examples:
 - A department has exactly one manager, and an employee can manage at most one department.
 - Specify (0, 1) for the participation of EMPLOYEE in MANAGES.
 - Specify (1, 1) for the participation of DEPARTMENT in MANAGES.
 - An employee works for exactly one department, and a department should have one or more employees.
 - Specify (1, 1) for the participation of EMPLOYEE in WORKS_FOR.
 - Specify (1, N) for the participation of DEPARTMENT in WORKS_FOR.

34

34

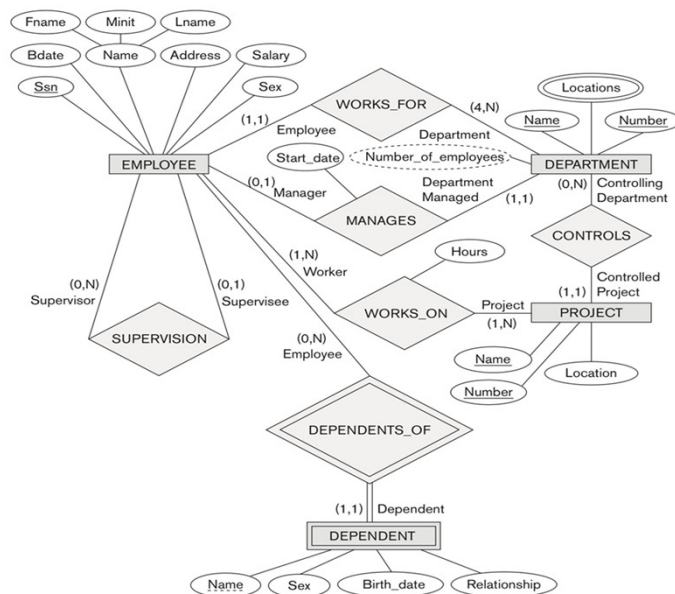
(min, max) Notation for Relationship Constraints



35

35

COMPANY ER Diagram Using (min, max) Notation

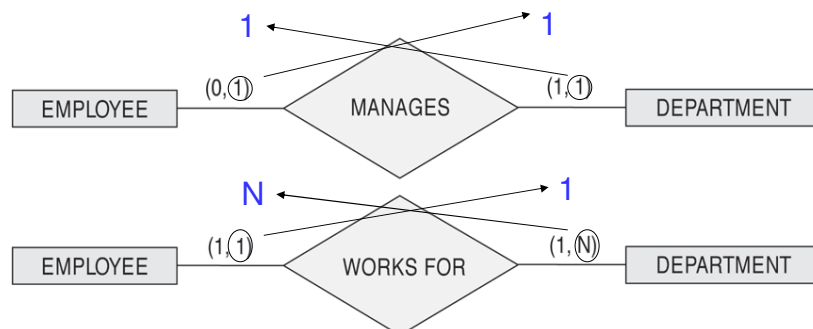


36

36

Conversion from (min, max) Notations to a Cardinality Ratio Constraint

- From each (*min*, *max*) notation of the two participating entity types, pick up the *max* value and send it to the other side of the relationship type. Then, those two *max* values represent the cardinality ratio constraint of the relationship type.



37

37

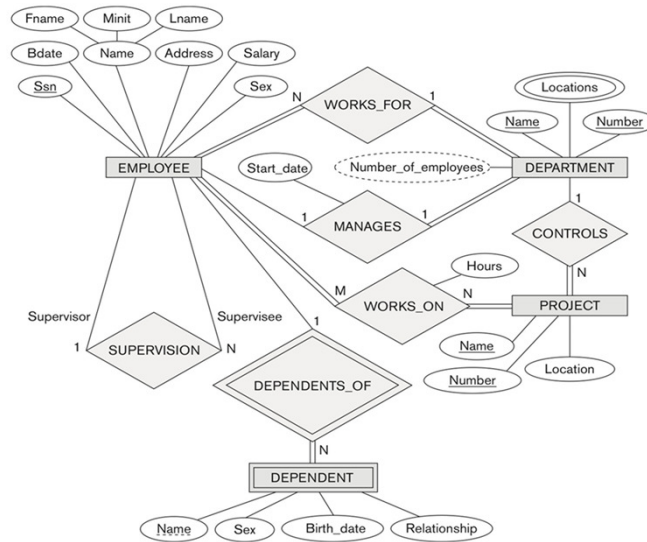
Attributes of Relationship Types

- A relationship type can have attributes.
- For example, the Hours attribute of **WORKS_ON** relationship type describes the weekly working hours of an employee on a project.
 - It has a value for each relationship instance of **WORKS_ON**, like $(e_1, p_3, 10)$.
 - A value of Hours depends on a particular (employee, project) combination. So, the value of relationship attribute is called **intersection data**.
 - On the other hand, Hours cannot be an attribute of **EMPLOYEE** or **PROJECT** entity type.

38

38

Example Attributes of Relationship Types: Hours of WORKS_ON and Start_date of MANAGES



39

39

Result of Mapping the COMPANY ER Schema into a Relational DB Schema

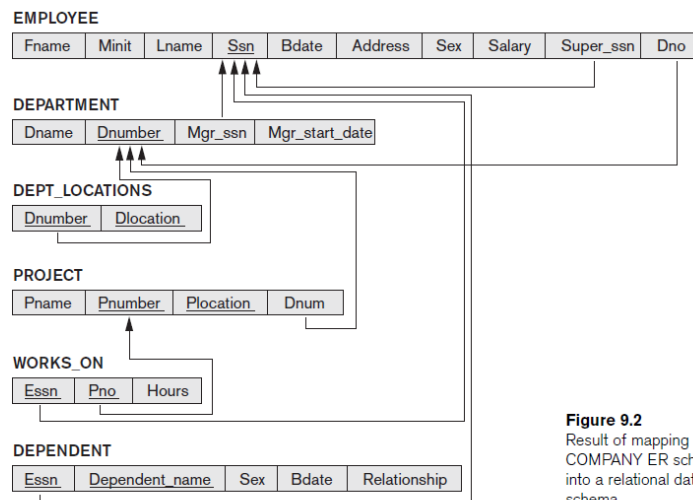


Figure 9.2
Result of mapping the
COMPANY ER schema
into a relational database
schema.

40

Weak Entity Types

- An entity type that does not have a key attribute is called **weak entity type**.
- Entities belong to a weak entity type are identified by being related to (i.e., associated with) specific entities from another entity type (called **identifying entity type** or **owner entity type**).
- The relationship type that relates a weak entity type to its owner entity type is called **identifying relationship type**.
- A **partial key** of the weak entity type is an attribute (a simple or a composite attribute) whose value can uniquely identify the weak entities that are related to (i.e., associated with) the same owner entity.

41

41

Weak Entity Types (cont'd)

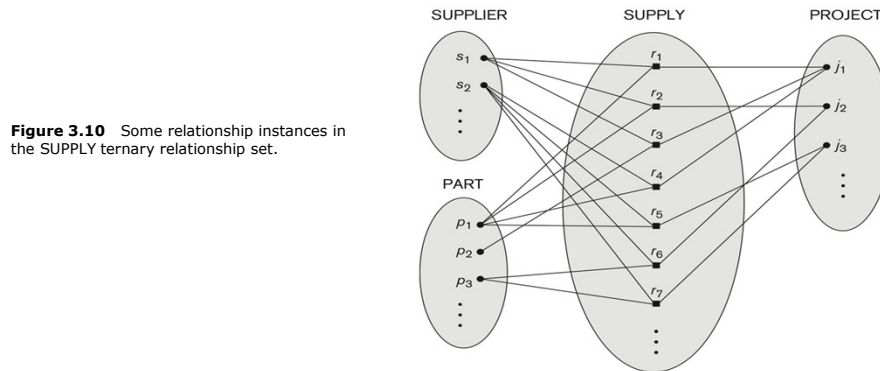
- **Example:**
 - **DEPENDENT** is a *weak entity type*.
 - A **DEPENDENT** entity is identified by the dependent's *Name* and the specific **EMPLOYEE** entity to which the **DEPENDENT** entity is related.
 - *Name* attribute of **DEPENDENT** is the *partial key*.
 - **EMPLOYEE** is its owner entity type via the identifying relationship type **DEPENDENTS_OF**.

42

42

Relationships of Higher Degree

- Relationship types of degree 2 are called *binary relationship types*.
- Relationship types of degree 3 are called *ternary relationship types*, and of degree n are called *n -ary relationship types*.



43

Discussion of n -ary relationships ($n > 2$)

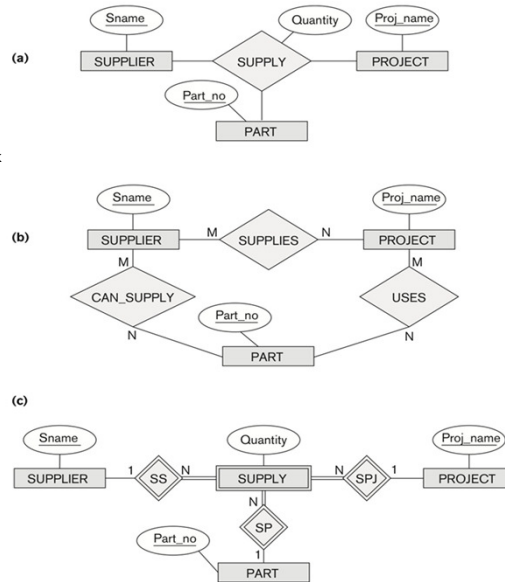
- In general, 3 binary relationships can represent different information than a single ternary relationship; i.e., they are not equivalent. (see Figures 17(a) and 17(b))
- In some cases, a ternary relationship type can be represented as a weak entity type if the data model allows a weak entity type to have multiple identifying relationships (and hence multiple owner entity types). (see Figure 17(c))
 - But this is much more complex and less clear than a ternary relationship type.
- Constraints are harder to specify for higher degree relationships ($n > 2$) than for binary relationships.
 - (min, max) notation can be used for each participating entity type.

44

44

Example of a ternary relationship

Figure 3.17 Ternary relationship types.
(a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.



45

Summary of Notations for ER Diagrams

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1:N for $E_1 : E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

46

Alternative Diagrammatic Notation

- ER diagram is one popular example of displaying a conceptual schema.
- Many other notations are used in various database design and modeling tools.
 - *Appendix A* illustrates some of the alternative notations that have been used.
- **Unified Modeling Language (UML) class diagram** is another conceptual representation that is used in several commercial design tools.

47

47

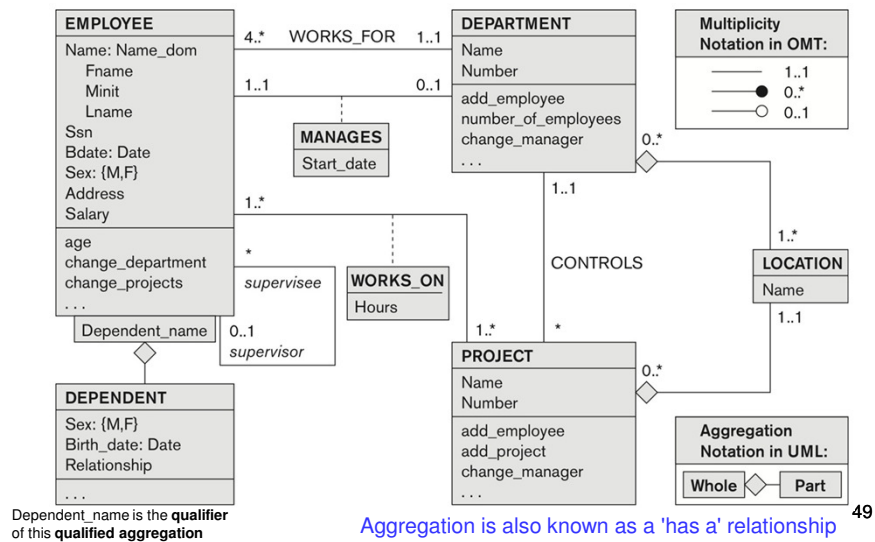
UML Class Diagram

- Represents **classes** (similar to entity types) as **boxes** with three sections:
 - Top section includes class (i.e., entity type) name.
 - Second section includes attributes.
 - Third section includes class operations. (Note: operations are not specified in basic ER model.)
- **Associations** (corresponding to relationship types) are represented as lines connecting the classes.
- Used in database design and object-oriented software design.
- UML has many other types of diagrams for software design: e.g., component diagram, activity diagram, communication diagram, etc.

48

48

UML Class Diagram for COMPANY Database Schema



49

Other alternative diagrammatic notations

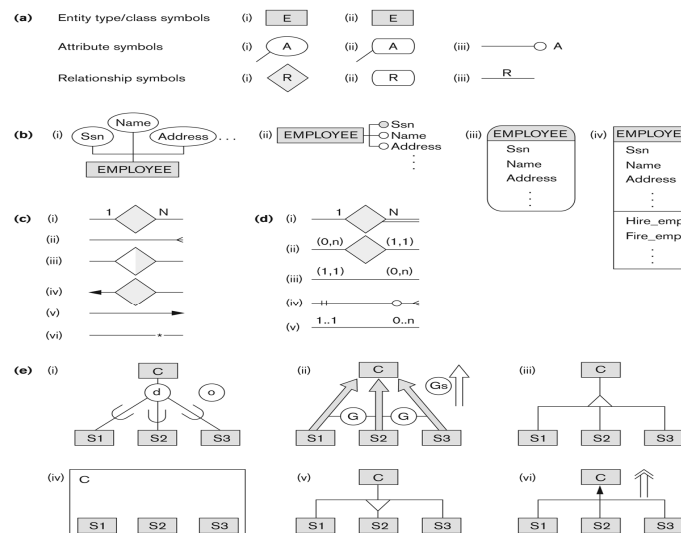


Figure A.1
Alternative notations. (a) Symbols for entity type/class, attribute, and relationship. (b) Displaying attributes. (c) Displaying cardinality ratios. (d) Various (min, max) notations. (e) Notations for displaying specialization/generalization.

50

50

ER Modeling Practice

Draw an ER diagram describing each of the following problems, and clearly state any assumptions that you make.

1. **University DB problem:** Convert the University relational database schema into an equivalent ER diagram and constraints that resulted in this schema (i.e., inverse schema mapping).

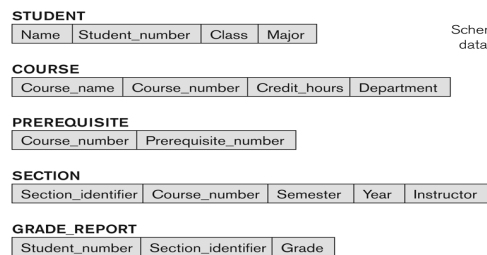


Figure 2.1
Schema diagram for the database in Figure 1.2.

51

51

ER Modeling Practice (cont'd)

2. **Person-Vehicle problem:** A person, identified by a PERSON-ID, can own any number of vehicles. Each vehicle has an unique VEHICLE-ID number and a MAKE (such as GM, Ford, Toyota, etc), and it is registered in any one of the states identified by STATE-NAME. A registration number (REG-NO) and registration TERMINATION-DATE are of interest, and so is the address of registration-office (REG-OFFICE-ADDRESS) in each state.

52

52

ER Modeling Practice (cont'd)

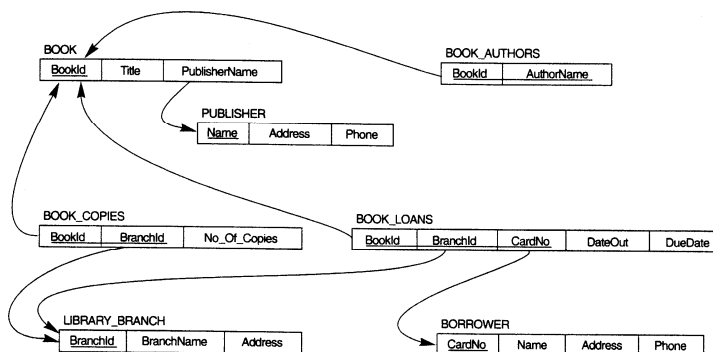
3. **Hospital problem:** Each person, identified by a PERSON-ID, can visit any number of hospitals as a patient. Each hospital has a number of doctors of different specialties, and suppose that each patient has a designated doctor (i.e., primary doctor) at each hospital. Naturally, each doctor can also be a patient at any hospital.

53

53

ER Modeling Practice (cont'd)

4. Consider the following relational database schema for a library. Convert the schema into an ER diagram and constraints that resulted in this schema (i.e., inverse schema mapping). Try to be as precise as possible in specifying the constraints.



54

54